



# From Expert Practices to Intelligent Agents: Autonomy in Interactive Video Retrieval

Minh-Quan Ho-Le<sup>1,2(✉)</sup> , Duy-Khang Ho<sup>1,2</sup> , Tu V. Ninh<sup>1,2,3</sup> ,  
Cathal Gurrin<sup>3</sup> , and Minh-Triet Tran<sup>1,2(✉)</sup> 

<sup>1</sup> University of Science, VNU-HCM, Ho Chi Minh City, Vietnam  
hlmq@selab.hcmus.edu.vn

<sup>2</sup> Vietnam National University, Ho Chi Minh City, Vietnam

<sup>3</sup> Dublin City University, Dublin, Ireland

**Abstract.** The Video Browser Showdown (VBS) is an international competition evaluating systems for interactive video retrieval across large datasets. In VBS 2026, we present **SnapMind**, the next generation of SnapSeek, which introduces an LLM-based Planner acting as an AI agent to support both expert and novice users. Given a query and the registry of retrieval components (e.g., text, image, OCR, ADL, object), the Planner generates candidate execution plans describing how these components can be combined. Users may select, edit, or discard plans, while the system executes them incrementally with normalization and fusion strategies. By embedding this AI agent into the loop, the system demonstrates agentic AI behavior: it can propose strategies, adapt to feedback, with the user controlling its participation via three autonomy levels. This design helps novices transform vague ideas into executable retrieval strategies while allowing experts to retain fine-grained control.

**Keywords:** Interactive Video Retrieval · LLM-based Planner · Agentic AI

## 1 Introduction

As video archives grow in scale and diversity, interactive multimedia retrieval has become crucial. The Video Browser Showdown (VBS) [12] benchmarks systems across known-item search (KIS), ad-hoc search, and question answering, in both expert and novice sessions, which highlights not only model quality but also usability and explainability. Our earlier systems, SnapSeek [4, 5], addressed these needs through multimodal indexing, scene sequencing, sketch-based search, and interactive feedback. While effective for experts, novice users are less familiar with the system and often struggle to select suitable modalities or perform basic interactions, revealing a persistent usability gap in VBS systems.

We introduce **SnapMind**, the fourth major iteration. Moving from pure seeking to mindful guidance, we add an LLM-based Planner - an AI agent that reads a component registry and proposes multiple execution plans (component

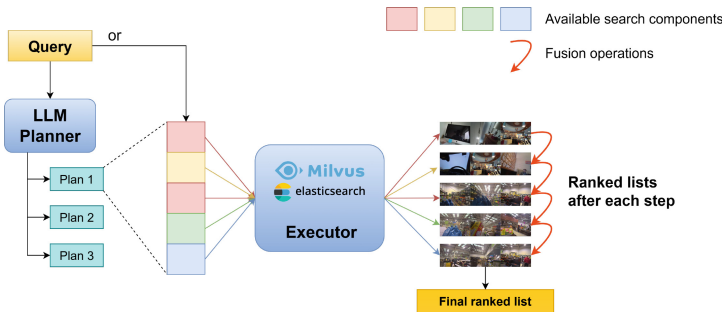
sequences, weights, top- $k$ , fusion). Users can inspect, select, and refine plans (insert/swap/delete components, change fusion operations, reweight) or ignore them, preserving expert control while scaffolding novice intent. **SnapMind** executes them atop SnapSeek 2.0’s incremental fusion engine, where component results are normalized, and merged via one of three rank-aggregation methods (CombSUM, CombMNZ, or RRF). Each item retains a `source_contrib` vector for explainability. In addition, an audit trail records plans, parameters, per-step ranked lists, and edits while early stopping halts when rankings stabilize; together, these yield efficiency, transparency, and reproducibility.

## Contributions

1. An **LLM-based Planner** (AI agent) that turns vague queries into candidate retrieval strategies from a fixed registry.
2. Seamless integration that enables **agentic behavior**: proposing, adapting to edits, and balancing autonomy (Guide/Assist/Auto).
3. Extended **fusion and explainability**: per-item source attribution and a stepwise audit trail for transparent, reproducible runs.

**Related Work.** Prior VBS systems combine multimodal indexing with interactive tools for composition and browsing [1, 3, 6, 7, 9–11]. In parallel, LLMs act as planners for vision pipelines, orchestrating tools for video QA and retrieval [2, 8, 13–15]. **Our difference** is an auditable, registry-bounded Planner that proposes editable, multi-step strategies and executes them with incremental fusion - bridging novice guidance and expert control within VBS. Together, these advances help systems reason about strategies, support users across expertise levels, and deliver explainable results for the competitive setting of VBS 2026.

## 2 Methodology



**Fig. 1.** Overall architecture of SnapMind, illustrating the LLM-based Planner, execution of retrieval components, and incremental fusion of results into the final ranked list.

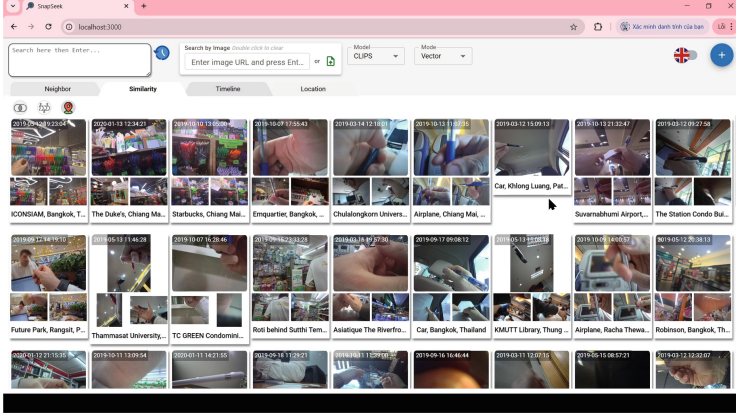


Fig. 2. SnapMind’s user interface.

## 2.1 Architecture Overview

We address interactive multimedia retrieval with an *incremental fusion architecture*, as described in Fig. 2. A query  $q$  (and optional context  $C$ ) is converted by an **LLM Planner** into a sequence of component calls from the **Component Registry** (§2.2). Each step yields a candidate list  $L^{(t)}$ , executed by the **Executor** and merged with the aggregated ranked list from previous step  $R^{(t-1)}$  by some fusion operations into  $R^{(t)}$ . A **UI Loop** governs interaction under a chosen *mode* (§2.5). Components are independent; integration happens solely through score fusion, therefore improves modularity, explainability, and mid-session adaptability.

## 2.2 Component Registry $\mathcal{R}$

$\mathcal{R}$  defines the contract between the Planner and components via query-agnostic metadata:

- `tool_id` (e.g., `txt.retriever.embedder.clipL:v1`, `ocr.bow:v1`)
- `modality`: `text`, `image`, `OCR`, `color`, `object`, `ADL`
- `score_range`, optional `calibration` for normalization
- `default_topk`, `default_weight` for selection

The Planner must select from  $\mathcal{R}$  only, ensuring reproducible, bounded, and safe execution.

## 2.3 Search Components

**SnapMind** uses components from  $\mathcal{R}$  with standardized I/O so the Planner can reason uniformly. Each component supports up to three fusion operations:

- **Search:** query the database, then fuse results into the cumulative ranking.
- **Rerank:** rescore the current list and fuse updated scores.
- **Filter:** drop items failing a criterion (e.g., keyword, threshold) while preserving order.

*Indexing.* We employ two schemes: (1) **dense embeddings** (OpenCLIP ViT-H/14) in Milvus for semantic/visual retrieval; (2) **keyword indexes** in Elasticsearch for symbolic queries (OCR text, object labels, colors, ADL tags) with fuzzy matching.

*Available Components.* Table 1 lists components and their supported fusion operations. Dense-embedding (*Semantic Text*, *Image*) support *Search/Rerank*; keyword-based (*OCR*, *Color*, *Object*, *ADL*) additionally support binary *Filter*. Global components act on frame-level metadata; local ones on region-level descriptors.

**Table 1.** Supported fusion operations and indexing types of registered components.

| Component       | Search | Rerank | Filter | Index Type   | Scope  |
|-----------------|--------|--------|--------|--------------|--------|
| Semantic Text   | ✓      | ✓      |        | Dense vector | Global |
| Image           | ✓      | ✓      |        | Dense vector | Global |
| OCR             | ✓      | ✓      | ✓      | Keyword      | Local  |
| Color (Global)  | ✓      |        | ✓      | Keyword      | Global |
| Color (Local)   | ✓      | ✓      | ✓      | Keyword      | Local  |
| Object (Global) | ✓      |        | ✓      | Keyword      | Global |
| Object (Local)  | ✓      | ✓      | ✓      | Keyword      | Local  |
| ADL (Activity)  | ✓      |        | ✓      | Keyword      | Global |

Each component has fixed cost/latency, enabling budget-aware plan selection.

## 2.4 LLM Planner

Given  $(q, C)$  and  $\mathcal{R}$ , the LLM Planner outputs:

1. A **plan**  $\pi = \langle S_1, \dots, S_T \rangle$  as an ordered component sequence.
2. Per-component **weights**  $w_{1:T}$ , **top- $k$**  budgets  $k_{1:T}$ .
3. A global **rank aggregation method**  $F \in \{\text{CombSUM}, \text{CombMNZ}, \text{RRF}\}$ .
4. Component-level **query rewrites** (synonyms, constraints, negatives).
5. A **mode protocol** (§2.5) specifying who executes each step and pause points.

**Example (Query).** “Time-lapse shot of an Asian woman and man on a bench under a tree, both facing the camera; grass around; man with black backpack, woman with leather shoulder bag; both wear sunglasses.”

1. **Step 1. Semantic Text Search:** dense retrieval for “man and woman on a bench under a tree.”

2. **Step 2. Object Filter (global):** keep person, bench, tree.
3. **Step 3. Color Filter (global):** prefer dominant green and turquoise.
4. **Step 4. Semantic Text Rerank:** emphasize “sunglasses, backpack, shoulder bag.”
5. **Step 5. Color Rerank (local):** match green grass/brown bench in lower-central regions.

The Planner does not score; it orchestrates components, rewrites queries for recall, enforces early stopping, and logs an auditable trail.

## 2.5 Modes of Interaction

We offer three modes that trade off autonomy and user effort:

**Guide (Low Autonomy).** Planner outputs *instructions*; the user executes every step (typing queries, sketching, triggering tools). After each step, the user may refine (e.g., “more red,” “dog bottom-right”), and the Planner revises the remaining plan. Progress is strictly step-by-step.

**Assist (Medium Autonomy).** Planner renders steps as *buttons*. The user chooses which to run; the system executes automatically. After each step, the user can edit weights, fusion, top- $k$ , or order; the Planner updates  $\pi$ ,  $w$ , and  $F$ . The loop pauses after every step.

**Auto (High Autonomy).** Planner provides an *Approve* button; once approved, the system runs the full plan. Early stopping triggers if rankings stabilize (tolerance  $\varepsilon$ ); otherwise the final fused ranking is returned.

## 2.6 Full Pipeline

The full interaction pipeline of the **SnapMind** system is detailed in Algorithm 1. The system first proposes candidate plans. It then executes the user-selected plan step by step. At each approved step  $t$ , the chosen component  $S_t$  calls *ExecuteStep* (Algorithm 2) and returns a candidate list with raw scores. Scores are normalized using the component’s calibration (e.g., logistic) or, if absent, by min-max. Items not returned at step  $t$  receive zero.

After normalization, the system incrementally fuses the per-step scores with the running aggregate using one of three rank aggregation methods: CombSUM, CombMNZ, or RRF. This produces an updated ranking list  $R^{(t)}$ . The list is truncated to the step’s top- $k$ , and the system emits the ranking with per-item `source_contrib` attributions for explainability. Mode protocols control pacing: *Guide* and *Assist* pause for user input, while *Auto* proceeds automatically. Early stopping applies when the top- $k$  stabilizes (Jaccard) and  $\Delta\text{NDCG}@K$  drops

below a threshold. The final result  $R^*$  includes an audit trail  $\mathcal{T}$  that logs component IDs, weights, fusion operations, top- $k$  values, and any user or planner edits. This supports reproducibility, fine-grained analysis, and controlled ablations.

---

**Algorithm 1: Full Pipeline**


---

**Input** : Input query  $q$ , context  $C$  (optional), component registry  $\mathcal{R}$ , mode  $m \in \{\text{Guide, Assist, Auto}\}$ , weights  $w_{1:T}$ , per-source top- $k$   $k_{1:T}$ , global aggregation method  $F$ , flag for user approval *approved*.

**Output** : final ranked list  $R^*$ , audit trail  $\mathcal{T}$

- 1 **Phase 1: Plan proposal and selection (common to all modes).**
- 2  $\Pi \leftarrow \text{ProposePlans}(q, C, \mathcal{R})$ ;
- 3  $(\Pi)$ ;
- 4  $\pi \leftarrow \text{UserSelectPlan}(\Pi)$ ;
- 5 **Phase 2: Execution of the locked plan.**
- 6  $L^{(0)} \leftarrow []$ ;  $R^{(0)} \leftarrow []$ ;  $t \leftarrow 0$ ;  $\mathcal{T} \leftarrow []$ ;
- 7 *approved*  $\leftarrow \text{False}$
- 8 **foreach**  $(S_t, q_t, w_t, k_t) \in \pi$  **do**
- 9      $t \leftarrow t + 1$ ;
- 10    **if**  $(m = \text{Auto} \wedge t = 1) \vee (m = \text{Assist}) \vee (m = \text{Guide})$  **then**
- 11      $\text{approved} \leftarrow \text{GetUserApproval}(S_t, q_t, k_t)$
- 12    **if**  $m = \text{Auto} \wedge t = 1 \wedge \neg \text{approved}$  **then break**;
- 13    **if** *approved* **then**
- 14      $L^{(t)} \leftarrow \text{ExecuteStep}(m, t, S_t, q_t, k_t)$ ;
- 15     **if**  $|L^{(t)}| = 0$  **then**
- 16         append  $\langle S_t, \text{empty} \rangle$  to  $\mathcal{T}$ ;
- 17         **continue**
- 18      $L^{(t)} \leftarrow \text{Normalize}(L^{(t)}, \mathcal{R}[S_t].\text{calibration})$ ;
- 19      $L^{(t)} \leftarrow \text{Fuse}(L^{(t)}, R^{(t-1)}, w_t, F)$ ;
- 20      $R^{(t)} \leftarrow \text{Cutoff}(R^{(t)}, k_t)$ ;
- 21      $R^{(t)}$  with per-item source\_contrib;
- 22     append  $\langle S_t, w_t, k_t, L^{(t)}, R^{(t)} \rangle$  to  $\mathcal{T}$ ;
- 23    **if**  $\text{UserRequestedEdits}()$  **then**
- 24          $(\pi, w, k, F) \leftarrow \text{AdjustBasedOnUserEdits}(\pi, w, k, F, R^{(t)})$  ;
- 25    **if**  $m = \text{Auto} \wedge \text{EarlyStop}(R^{(t-1)}, R^{(t)}, \varepsilon)$  **then**
- 26          $R^* \leftarrow R^{(t)}$ ;
- 27         **break**
- 27    **if**  $m = \text{Assist} \vee m = \text{Guide}$  **then** *approved*  $\leftarrow \text{False}$  ;
- 28 **if**  $R^*$  not set **then**  $R^* \leftarrow R^{(t)}$  ;

---

**Algorithm 2:** ExecuteStep

---

```

1 Function ExecuteStep( $m, t, S_t, q_t, k_t$ )
2    $L^{(t)} \leftarrow []$ ;
3   if  $m = \text{Guide}$  then
4     EmitInstruction( $S_t$ );
5      $L^{(t)} \leftarrow \text{RunSearch}(S_t, q_t, k_t)$ ;
6   else if  $m = \text{Assist}$  then
7     ShowActionButton( $S_t$ );
8     if UserClicked() then
9        $L^{(t)} \leftarrow \text{RunSearch}(S_t, q_t, k_t)$ ;
10  else if  $m = \text{Auto}$  then
11     $L^{(t)} \leftarrow \text{RunSearch}(S_t, q_t, k_t)$ ;
12  return  $L^{(t)}$ ;

```

---

### 3 Conclusion

In summary, we present **SnapMind** - an interactive multimedia retrieval system with an LLM Planner that composes and executes multi-step strategies over a fixed, auditable registry. An incremental fusion loop (normalization, rank aggregation) with per-item `source_contrib` and an audit trail, plus *Guide/Assist/Auto* modes and budgeted early stopping, yields accountable, novice-friendly retrieval that scales across modalities and domains.

### References

1. Amato, G., et al.: The visione video search system: exploiting off-the-shelf text search engines for large-scale video retrieval. *J. Imaging* **7**(5) (2021). <https://doi.org/10.3390/jimaging7050076>, <https://www.mdpi.com/2313-433X/7/5/76>
2. Fan, Y., et al.: Videoagent: a memory-augmented multimodal agent for video understanding. In: Leonardis, A., Ricci, E., Roth, S., Russakovsky, O., Sattler, T., Varol, G. (eds.) *Computer Vision – ECCV 2024*. ECCV 2024. LNCS, vol. 15080, pp. 75–92. Springer, Cham (2025). [https://doi.org/10.1007/978-3-031-72670-5\\_5](https://doi.org/10.1007/978-3-031-72670-5_5)
3. Hezel, N., Schall, K., Jung, K., Barthel, K.U.: Efficient search and browsing of large-scale video collections with vibro. In: Hór Jónsson, B., et al. *MultiMedia Modeling*. MMM 2022. LNCS, vol.13142, pp. 487–492. Springer, Cham (2022). [https://doi.org/10.1007/978-3-030-98355-0\\_43](https://doi.org/10.1007/978-3-030-98355-0_43)
4. Ho-Le, M.Q., et al.: Snapseek: an interactive lifelog acquisition system for lsc’24. In: *Proceedings of the 7th Annual ACM Workshop on the Lifelog Search Challenge*, pp. 24–29. LSC ’24, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3643489.3661116>
5. Ho-Le, M.Q., et al.: SnapSeek 2.0 at video browser showdown 2025. In: Ide, I., et al. (eds.) *MultiMedia Modeling*. MMM 2025. LNCS, vol. 15524, pp. 339–346. Springer, Singapore (2025). [https://doi.org/10.1007/978-981-96-2074-6\\_41](https://doi.org/10.1007/978-981-96-2074-6_41)
6. Le, T.K., Ninh, V.T., Tran, M.K., Healy, G., Gurrin, C., Tran, M.T.: Avseeker: an active video retrieval engine at vbs2022. In: Hór Jónsson, B., et al. *MultiMedia Modeling*. MMM 2022. LNCS, vol. 13142. Springer, Cham (2022). [https://doi.org/10.1007/978-3-030-98355-0\\_51](https://doi.org/10.1007/978-3-030-98355-0_51)

7. Nguyen, T.N., Quang, L.M., Healy, G., Nguyen, B.T., Gurrin, C.: Videoclip 2.0: an interactive clip-based video retrieval system for novice users at vbs2024. In: Rudinac, S., et al. (eds.) *MultiMedia Modeling, LNCS*, pp. 394–399. Springer, Cham (2024). [https://doi.org/10.1007/978-3-031-53302-0\\_37](https://doi.org/10.1007/978-3-031-53302-0_37)
8. Rosa, K.D.: Smart routing for multimodal video retrieval: when to search what. *arXiv preprint arXiv:2507.13374* (2025)
9. Schall, K., Hezel, N., Barthel, K.U., Jung, K.: Optimizing the interactive video retrieval tool vibro for the video browser showdown 2024. In: Rudinac, S., et al. (eds.) *MultiMedia Modeling. MMM 2024. LNCS*, vol. 14557, pp. 364–371. Springer, Cham (2024). [https://doi.org/10.1007/978-3-031-53302-0\\_33](https://doi.org/10.1007/978-3-031-53302-0_33)
10. Schall, K., Hezel, N., Jung, K., Barthel, K.U.: Vibro: video browsing with semantic and visual image embeddings In: Dang-Nguyen, D.T., et al. (eds.) *MultiMedia Modeling. MMM 2023. LNCS*, vol. 13833. Springer, Cham (2023). [https://doi.org/10.1007/978-3-031-27077-2\\_56](https://doi.org/10.1007/978-3-031-27077-2_56)
11. Schoeffmann, K., Nasirihaghighi, S.: Divexplore at the video browser showdown 2024. In: Rudinac, S., et al. (eds.) *MultiMedia Modeling. MMM 2024. LNCS*, vol. 14557, pp. 372–379. Springer, Cham (2024). [https://doi.org/10.1007/978-3-031-53302-0\\_34](https://doi.org/10.1007/978-3-031-53302-0_34)
12. Schöffmann, K., Bailer, W.: Video browser showdown. *SIGMultimedia Rec.* 4(2), 1–2 (2012). <https://doi.org/10.1145/2350204.2350205>
13. Shen, Y., Song, K., Tan, X., Li, D., Lu, W., Zhuang, Y.: Hugginggpt: solving ai tasks with chatgpt and its friends in hugging face. *Adv. Neural. Inf. Process. Syst.* **36**, 38154–38180 (2023)
14. Vuong, G.H., et al.: Viewsinsight2.0: enhancing video retrieval for vbs 2025 with an automatic query generator powered by large language models. In: Ide, I., et al. (eds.) *MultiMedia Modeling, LNCS*, pp. 371–377. Springer, Singapore (2025). [https://doi.org/10.1007/978-981-96-2074-6\\_45](https://doi.org/10.1007/978-981-96-2074-6_45)
15. Zhi, Z., Wu, Q., Li, W., Li, Y., Shao, K., Zhou, K., et al.: Videoagent2: enhancing the llm-based agent system for long-form video understanding by uncertainty-aware cot. *arXiv preprint arXiv:2504.04471* (2025)